

Lesson 10 · Jev and System One models

PDF: [this lesson](#) · [slide deck](#) · **Install (Linux · macOS · Windows):** [guide](#)

Part of [local-ai-lab](#) - a hands-on course for building local AI.

Interactive version (slides):

<https://nikolareljin.github.io/local-ai-lab/lesson-10-jev-system-one.html> **Read it locally (no GitHub Pages):** `./run -l 10 lesson` **Course home:** <https://nikolareljin.github.io/local-ai-lab/>
Source: <https://github.com/nikolareljin/local-ai-lab> **Author:** [Nik Reljin](#) **Time:** ~60 min ·
Prerequisites: Lesson 1 (Lessons 4, 5 and 9 helpful) · full objectives in [SYLLABUS.md](#)

Lessons: [1 · RAG](#) → [2 · MCP](#) → [3 · Hybrid retrieval](#) → [4 · RAG safety](#) → [5 · RAG evaluation](#) → [6 · Repo assistant](#) → [7 · LangChain](#) → [8 · LangGraph](#) → [9 · Ollama tools](#) → **10 · Jev / System One (you are here)** → [11 · Semantic Kernel](#) → [12 · Bedrock Agents](#) → [13 · Google ADK](#) → [14 · AI-assisted testing](#) → [15 · AI code review](#) → [16 · Docs from changes](#)

Status: working demo, special lesson. Runnable in **Python, Node.js and C#**. The demo installs nothing and needs no model: it replays answers recorded from real runs. The live actions need Ollama; the hosted Jev needs a TypeSafe API key.

First - what is a System One model?

Every lesson so far used a model that **writes text**. Even in Lesson 9, where the model chose a tool, it did so by writing JSON that your code then had to check.

A **System One model** does not write. You give it one input (the **state**) and a list of typed **questions**, and it returns **a probability for every possible answer to every question**, in one pass. Nothing to parse, nothing to validate: the answer cannot be anything but one of the options you defined.

```
{"state": "I was charged twice. Please refund one.",  
"questions": {"refund_request": {"type": "noul", "instructions": "Does the customer ask for money back?"}}}
```

```
{"answers": {"refund_request": {"type": "noul", "noul": 0.97}}}
```

Jev is TypeSafe's System One model ([announcement](#), [docs](#)). The name comes from "System 1" thinking: the fast, intuitive judgement, as opposed to slow step-by-step reasoning. Three question types cover most decisions software makes:

Type	Asks	Answer
<code>noul</code>	a yes/no question	P(yes), a number from 0 to 1
<code>choice</code>	pick one of 2-255 labels	a probability per label, the top label, a confidence

score	a rating on 2-10 ordered levels	a probability per level, a weighted score, a confidence
-------	---------------------------------	---

Jev vs an LLM

	LLM (Lessons 1-9)	System One model (Jev)
Output	text, token by token	a probability per option, one pass
Typed?	only if you validate what it wrote	always: the options are the output
Probabilities	none you can trust, usually	calibrated, per TypeSafe: 0.8 should be right 8 times in 10
Many questions	one long answer, or one call each	up to 64 questions in one request
Explains itself	yes	no
Writes, summarises, converses	yes	no
Speed (TypeSafe's figures)	seconds	70-500 ms

The last row is TypeSafe's claim, not a measurement made here. Everything measured in this lesson says so where it appears.

What exists today (checked 2026-10-02)

What	Where it runs	Used in this lesson
TypeSafe Jev (jev-1.13.0, route jev-latest)	TypeSafe's servers only; early access	yes, opt-in, fake data only
typesafe-sdk (PyPI, 0.7.2)	your machine, calls the API	yes, pinned with sha256 hashes
@typesafe-ai/sdk (npm, 0.6.0)	your machine, calls the API	yes, pinned, provenance-attested
A local Jev	does not exist	-
Community look-alikes on GitHub	your machine	no : unsigned, no releases

There is **no local Jev**. TypeSafe ships the model only as a hosted API. So this lesson does two things:

1. Calls the **real Jev** through TypeSafe's official SDK, when you bring a key.
2. Builds a **local Jev-like adapter**: a small server that speaks the same API and answers with a local model through Ollama. It is labelled **simulated** everywhere it appears. It shows how the idea works and what it costs to fake it - not how good Jev is.

A name to watch: `pip install qev` does **not** install anything from TypeSafe. That PyPI name belongs to an unrelated project. The official Python package is `typesafe-sdk`.

What you'll learn

- What a typed, probabilistic decision is, and why it is a different tool from a chat model.
- The System One wire format by hand: one POST, three question types.
- The official SDK in Python and Node.js, and the same call from C# with `HttpClient`.
- How to make a local LLM answer **like** a System One model (one token, letter log-probabilities), and where that imitation falls short.
- Why the **policy** - the code that turns probabilities into actions - must be yours.
- How to score decision engines honestly: accuracy, typed-answer rate, **Brier score** (calibration), and the business metric (wrong and missed pages).
- When to use a System One model, an LLM, or plain rules.

The demo

A fictional company, **Larkspur Cloud**, gets support tickets. Each one needs four decisions:

Question	Type	Options
queue	choice	billing, technical, account, sales, trust_safety
urgency	score	none, low, medium, high, critical
refund_request	noul	yes / no
needs_human	noul	yes / no

The questions are in `data/questions.json`, written in TypeSafe's own request format. Then a policy of a few lines decides what happens to the ticket:

```
trust & safety likely (P >= 0.5)      -> escalate: trust & safety
technical and P(high or critical) >= PAGE -> page on-call
billing and P(refund) >= REFUND      -> draft refund for approval
needs a human, or queue confidence low -> human triage
otherwise                             -> auto-route
```

The data (all fake, all labelled)

`data/generate.py` writes three datasets from a fixed seed. Every record has the answer a careful human gave it, so any engine can be scored:

File	Records	Questions
<code>tickets.jsonl</code>	30 support tickets	queue, urgency, refund_request, needs_human

reviews.jsonl	15 product reviews	sentiment, refund_request, mentions_bug
incidents.jsonl	15 server alerts	severity, owner, customer_facing

About a third of the records are **traps**, each marked with a `trap` field: a ticket that uses the word "refund" but does not want one, sarcasm, a ticket in Spanish or German, a "bug" that is really a data leak, and one ticket that tries to order the triage system around - Lesson 4's prompt injection, aimed at a classifier this time. Names come from a fixed list, every email is @example.com, every IP address is from 192.0.2.0/24, which is reserved for documentation.

Four engines, one interface

Engine	What it is	Model calls per ticket
keywords	rules written from the question criteria, before reading any ticket	0
LLM writes JSON	a local chat model asked to write the four answers as JSON	1
Jev-like adapter	this lesson's server: a local model answering one question at a time	4
TypeSafe Jev	the real model, over the internet	1

All four produce a System One response, so the policy and the scorecard cannot tell them apart.

Run it

```
./run -l 10 demo           # Python: replay the recorded runs, print the scorecard
./run -l 10 --lang node demo # the same scorecard from Node.js
./run -l 10 --lang csharp demo # and from C#
./run -l 10                # the playground (needs Flask)
```

Read the output

Recorded on 2026-10-02 on an Intel Core i5-10310U laptop (8 threads, 19 GB RAM, no GPU), Ollama 0.20.4. The full output is in expected-output.txt.

Engine	Right of 30 (queue / urgency / refund / human)	Brier	Actions	Pages wrong / missed	s / ticket
keywords (rules)	20 / 12 / 25 / 25	0.633	15/30	5 / 3	0

LLM writes JSON (qwen3:1.7b)	18 / 14 / 28 / 10	0.833	11/30	3 / 1	8.5
Jev-like adapter (qwen3:1.7b)	23 / 8 / 27 / 23	0.619	20/30	4 / 1	17.4
Jev-like adapter (qwen3.5:4b)	27 / 10 / 29 / 24	0.381	25/30	1 / 1	61.9

TypeSafe Jev is not in the table: recording it needs an API key (`./run -l 10 record --backend typesafe`).

What the numbers say:

- **Same model, two ways of asking.** qwen3:1.7b asked to **write** JSON gets 11/30 actions right; asked one lettered question at a time it gets 20/30. Most of the difference is `needs_human` (10/30 vs 23/30): the written answers say "yes" on all 30 tickets, where the labels say yes on 10.
- **Every answer was a valid option** this time, for every engine (the `typed` column, 100%). The JSON arm still has nothing to put a threshold on: its probabilities are all 0 or 1.
- **Calibration is where the bigger model earns its time.** The 4B adapter's Brier score is 0.381, against 0.6-0.8 for the rest, and only for it does moving `PAGE` change anything: 3 wrong pages at 0.5, 1 at 0.7, and at 0.9 one more missed page.
- **Urgency is hard for everyone** (8-14 of 30). The labels judge impact, not tone - "URGENT!!!" about a chart colour is `low` - and no local engine learned that from one sentence.
- **The prompt injection (T-1021)** fooled the keyword rules, the JSON arm and the 1.7B adapter. Only the 4B adapter routed it to `technical` with `none` urgency, and with a confidence of 0.68 rather than 1.0.
- **Cost.** The adapter makes four model calls per ticket, so the 4B model takes about a minute per ticket on this CPU. Jev answers all four questions in one request; TypeSafe quotes 70-500 ms.

Concept 1 · The whole API is one POST

`python/systemone.py` builds the request, sends it with the standard library, and reads the answer:

```
POST https://api.typesafe.ai/v1/systemone
Authorization: Bearer <TYPESAFE_API_KEY>
{"model": "jev-latest", "state": "...", "questions": {"queue": {...}, "urgency": {...}}
```

`read_answers()` turns every answer into the same shape - `{"pick", "probs", "confidence"}` with labels, not wire keys - and reports any answer that does not match the question it was asked. With a real System One model that list stays empty. It is there because this lesson also scores

engines that can return anything.

`post()` refuses to send anything anywhere except `https://api.typesafe.ai` or a loopback address. A typo in `TYPESAFE_BASE_URL` cannot hand your key, or your tickets, to a stranger.

Concept 2 · The official SDK - the same code for Jev and for the local adapter

```
./run -l 10 install-sdk          # once: typesafe-sdk 0.7.2 into lessons/10-*/.venv-sdk
export TYPESAFE_API_KEY=...      # from https://console.typesafe.ai
./run -l 10 sdk "I was charged twice, please refund one"
```

`python/sdk_example.py` is the whole integration:

```
client = TypeSafeClient()        # reads TYPESAFE_API_KEY and TYPESAFE_BASE_URL
result = client.system_one(text, {
    "queue": Choice(instructions="Which team queue should handle this support ticket?",
                    criteria={"billing": "Charges, invoices, refunds", ...}),
    "urgency": Score(instructions="How urgent is it for the business?",
                    criteria=["none", "low", "medium", "high", "critical"]),
    "refund_request": Noul(instructions="Does the customer ask for money back?"),
})
result.choices["queue"].probabilities    # {"billing": 0.91, "technical": 0.03, ...}
```

Node.js (`node/sdk_example.mjs`):

```
import { TypeSafeClient, choice, noul, score } from "@typesafe-ai/sdk";
const client = new TypeSafeClient();
const { answers } = await client.systemOne({ state: text, questions: {
  queue: choice("Which team queue should handle this support ticket?", { billing: "Charges, invoices, refunds", /* ... */ }),
  urgency: score("How urgent is it for the business?", ["none", "low", "medium", "high", "critical"]),
  refund_request: noul("Does the customer ask for money back?"),
}});
```

C# has no official SDK, so `dotnet/SystemOne.cs` posts the same JSON with `HttpClient`. Point any of the three at the local adapter instead of TypeSafe by changing **one environment variable**:

```
./run -l 10 serve                # terminal 1: the adapter on 127.0.0.1:8765
export TYPESAFE_BASE_URL=http://127.0.0.1:8765    # terminal 2
export TYPESAFE_API_KEY=local      # the adapter ignores it; the SDK wants one
./run -l 10 sdk "Our API returns 502 for everyone"
```

How the SDKs were installed - and why that matters

Both SDKs are pinned, so what runs is what was reviewed:

- **Python:** `requirements-sdk.txt` lists every package with its sha256. `pip install --require-hashes` refuses a file whose hash differs. Change one hash and the install stops with `THESE PACKAGES DO NOT MATCH THE HASHES`.
- **Node.js:** `node/package.json` pins `@typesafe-ai/sdk` to an exact version, and `package-lock.json` records its integrity hash. `npm audit signatures` checks the registry signature and the provenance attestation: npm can show this package was built from `github.com/typesafe-ai/typesafe-sdk-js` by GitHub Actions.

Neither is needed for the demo or the tests. The SDK test is skipped when the SDK is absent.

Concept 3 · Making a local LLM answer like a System One model

python/local_adapter.py is a small server with the same endpoint. For each question it:

- 1. Writes a multiple-choice prompt - options lettered A, B, C... - with the ticket fenced as data.
- 2. Lets the model produce **one token**, at temperature 0.
- 3. Reads Ollama's **log-probabilities** for the top 20 candidates of that token, and gives each option the probability of its letter, renormalised.

That is a real probability for every option, from one short forward pass: the System One contract. What it does not have is Jev's training for the job:

- **Calibration.** A small model is often 100% sure and wrong. The Brier score shows it.
- **Cost.** One model call per question. Jev answers up to 64 questions in one pass.
- **Speed.** Seconds on a laptop CPU, per question.

The server binds to 127.0.0.1 and refuses anything else: it has no authentication.

Concept 4 · The model answers; your policy decides

python/policy.py has three thresholds - PAGE, REFUND, CONFIDENT - and no model. This is the most important file in the lesson:

- The model never chooses "page on-call". It says $P(\text{high or critical}) = 0.83$. Your code decides that 0.83 is enough.
- Moving a threshold changes behaviour **without retraining or re-prompting anything**. The demo prints the pages at PAGE = 0.5, 0.7 and 0.9.
- That only works if the probabilities mean something. Rules and JSON answers are always 0 or 1, so for them the knob does nothing.

Concept 5 · Scoring a decision engine

python/scorecard.py:

Metric	Answers the question
accuracy	how often is the top answer right?
typed	how often was the answer a valid option at all?
Brier score	when it says 0.9, is it right 9 times in 10? 0 is perfect; certain and wrong costs 2
actions	does the policy do the same thing it would do with the human labels?

wrong / missed pages	the cost you actually pay at 3 a.m.
----------------------	-------------------------------------

Accuracy alone would rank an engine that is confidently wrong next to one that knows when it is unsure. Only the second one lets a threshold do its job. Lesson 5 scored a RAG pipeline against a golden set; this is the same discipline for decisions.

Concept 6 · Recorded once, replayed everywhere

`./run -l 10 record --backend local` asks the engine about every ticket and saves the replies in `data/cassettes/`, with the date, the hardware and a digest of each request. `demo` replays them through the same parsing, policy and scorecard code, in all three languages. If a ticket or a question changes, its digest stops matching and the replay refuses instead of scoring a conversation that never happened (the Lesson 9 cassette idea). The adapter's prompt has a fingerprint too: change it and the recordings must be redone.

How-to

Run the local adapter and call it

```
ollama pull qwen3:1.7b
./run -l 10 serve --model qwen3:1.7b
# another terminal
curl -s http://127.0.0.1:8765/v1/systemone -H 'Content-Type: application/json' -d '{
  "model": "local", "state": "The dashboard is down again",
  "questions": {"urgent": {"type": "noul", "instructions": "Is this urgent?"}}}'
```

Ask about one ticket, with any engine

```
./run -l 10 ask "Our API returns 502 for everyone since 09:40" --backend local
./run -l 10 ask "Our API returns 502 for everyone since 09:40" --backend keywords
./run -l 10 ask "Our API returns 502 for everyone since 09:40" --backend llm-json
```

Score engines on your machine - local and TypeSafe side by side

```
ollama pull qwen3:1.7b
export TYPESAFE_API_KEY=... # optional; without it the TypeSafe row says "not run"
./run -l 10 live --backend keywords,llm-json,local,typesafe --limit 10
./run -l 10 live --backend local --model qwen3.5:4b --dataset incidents
```

`--backend` takes one engine or several, comma-separated. Every engine answers the same records in the same run and lands in one scorecard. An engine that cannot run - no API key, Ollama not started - gets a row saying why, and the others still run. With `typesafe`, the fake tickets leave your machine. The playground (`./run -l 10`) has a live switch for each: the local adapter and TypeSafe.

Record what the demo replays

```
./run -l 10 record --backend local --model qwen3:1.7b --hardware "my laptop"
./run -l 10 record --backend typesafe # needs TYPESAFE_API_KEY
```


Recordings are named per engine, model and dataset (`jev-like-qwen3-1.7b.json` for the tickets, `jev-like-qwen3-1.7b-reviews.json` for the reviews), so recording one never overwrites another. A run with `--limit N` writes `...-firstN.json`, which the demo does not read.

Record one model at a time with nothing else on the CPU. Two resident models on a 19 GB laptop got the Ollama service killed in Lesson 9.

Use your own data

- 1. Write records as JSON lines with `id`, `text`, `labels` (one label per question) and `trap` ("" for none).
- 2. Add a question set to `data/questions.json`.
- 3. `./run -l 10 live --backend local --dataset <name>` (add the name to the `--dataset` choices in `python/jev.py`).

When to use which

Use	When	Examples
Rules	the signal is a word or a number, and a miss is cheap	"invoice" -> billing; disk > 90% -> alert
System One model	a judgement over unstructured text, many items, typed output, a threshold	ticket routing, content moderation, lead scoring, fraud flags, alert severity, guarding an LLM's output
LLM	the output is text, or the steps are not known in advance	replies, summaries, explanations, tool-using agents (Lesson 9)
Both	an LLM acts, a System One model checks	Jev as the guard on Lesson 9's tool calls; Jev deciding whether a retrieved passage is an injection (Lesson 4)

Ideal applications share three properties: the answer is one of a known set, there are many items (map over a dataset, triage a queue), and a business rule needs a probability to compare against a threshold.

Poor fits: anything that must be written, anything that needs reasons a person can read, and anything a keyword already decides.

Python, Node and C

	Python	Node.js	C#
demo (replay)	<code>python/jev.py</code>	<code>node/jev.mjs</code>	<code>dotnet/Program.cs</code>

live call	python/systemone.py, official SDK	official SDK	HttpClient
adapter server	python/local_adapter.py	-	-
tests	python/test_jev.py	demo byte-diff	demo byte-diff

All three print the same scorecard, byte for byte.

Exercises

1. Drag `PAGE` in the playground until wrong pages hit zero. How many missed pages did that cost?
2. Add a question `language` (choice: en, es, de) to the ticket set and record it. Which engine handles it best?
3. Run `--dataset incidents` against the adapter with two model sizes. Is the bigger model better calibrated, or only more accurate?
4. Rewrite one keyword rule to fix trap T-1004. What does it break?
5. With a TypeSafe key: record `typesafe` and compare its Brier score with the adapter's.

From demo to production

- **Keep the policy in code, versioned and tested.** Thresholds are product decisions.
- **Watch calibration over time.** Re-score a labelled sample each week; a Brier score that drifts up is the early warning.
- **Know where the data goes.** A hosted model means the text leaves your machine. Use it on what you are allowed to send.
- **Log the probabilities, not only the action.** They are what you will tune against later.
- **Treat the input as untrusted.** A ticket can contain instructions; the model should never be the only thing between them and an action.

Next lesson

Lesson 11 · Microsoft Semantic Kernel (C#) → - the Lesson 9 agent rebuilt in .NET, where automatic function calling runs the loop for you.